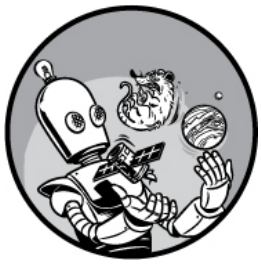
6

WRITING IN INVISIBLE INK



In the fall of 2012, the crime drama *Elementary* debuted on the CBS television network. A reimagining of the Sherlock Holmes mythos set in 21st-century New York, it starred Jonny Lee Miller as Holmes and Lucy Liu as his sidekick, Dr. Joan Watson. In a 2016 episode ("You've Got Me, Who's Got You?"), Morland Holmes, Sherlock's estranged father, hires Joan to find a mole in his organization. She quickly solves the case by discovering a Vigenère cipher in an email. But some fans of the show were dissatisfied: the Vigenère cipher is hardly subtle, so how could a man as intelligent as Morland Holmes miss finding it on his own?

In this project, you'll reconcile this dilemma using steganography, but not with a null cipher as in **Chapter 5**. To hide this message, you'll use a third-party module called `python-docx` that will allow you to conceal text by directly manipulating Microsoft Word documents using Python.

Project #12: Hiding a Vigenère Cipher

In the *Elementary* episode, Chinese investors hire Morland Holmes's consulting company to negotiate with the Colombian government for petroleum licenses and drilling rights. A year has passed, and at the last moment a competitor swoops in and clinches the deal, leaving the Chinese investors high and dry. Morland suspects betrayal by a member of his staff and asks Joan Watson to investigate alone. Joan identifies the mole by finding a Vigenère cipher in one of his emails.

SPOILER ALERT

The decrypted contents of the cipher are never mentioned, and the mole is murdered in a subsequent episode.

The *Vigenère cipher*, also known as the unbreakable cipher, is arguably the most famous cipher of all time. Invented in the 16th century by the French scholar Blaise de Vigenère, it is a polyalphabetic substitution cipher that, in the most commonly used version, employs a single keyword. This keyword, such as *BAGGINS*, is printed repeatedly over the plaintext, as in the message shown in **Figure 6-1**.

```
B A G G I N S B A G G I N S B A G G I
s p e a k f r i e n d a n d e n t e r
```

Figure 6-1: A plaintext message with the Vigenère cipher keyword BAGGINS *printed above*

A table, or *tableau*, of the alphabet is then used to encrypt the message. **Figure 6-2** is an example of the first five rows of a Vigenère tableau. Notice how the alphabet shifts to the left by one letter with each row.

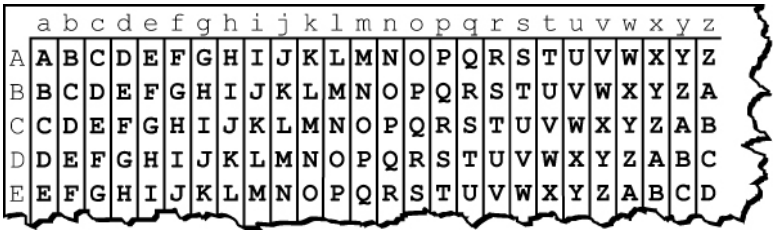


Figure 6-2: Portion of a Vigenère tableau

The keyword letter above the plaintext letter determines which row to use for the encryption. For example, to encrypt the *s* in *speak*, note that the keyword letter above it is *B*. Go down to the B row and read across to where the plaintext *s* is at the top of the column. Use the *T* at the intersection for the ciphertext letter.

Figure 6-3 shows an example of the full message encrypted with the Vigenère cipher. This kind of text would surely draw attention and become an object of scrutiny if it were visible in a document!

```
TPKGS SJJETJ IAV FNZKZ
```

Figure 6-3: A message encrypted with the Vigenère cipher

The Vigenère cipher remained unbroken until the mid-19th century, when Charles Babbage, inventor of the precursor to the computer, realized that a short keyword used with a long message would result in repeating patterns that could reveal the length of the keyword and, ultimately, the key itself. The breaking of the cipher was a tremendous blow to professional cryptography, and no significant advancements were made in the field during the Victorian era of the original Holmes and Watson.

The presence of this cipher is what causes “suspension of disbelief” issues with the *Elementary* episode. Why would an outside consultant be needed to find such a clearly suspicious email? Let’s see if we can come up with a plausible explanation using Python.

THE OBJECTIVE

Assume you are the corporate mole in the episode and use Python to hide a secret message summarizing bid details within an official-looking text document. Start with an unencrypted message and finish with an encrypted version.

The Platform

Your program should work with ubiquitous word-processing software, as the output needs to be sharable between different corporations. This implies use of the Microsoft Office Suite for Windows or compatible versions for macOS or Linux. And restricting the output to a standard Word document makes hardware issues Microsoft's responsibility!

Accordingly, this project was developed with Word 2016 for Windows, and the results checked with Word for Mac v16.16.2. If you don't have a license for Word, you can use the free Microsoft Office Online app, available at **https://products.office.com/en-us/office-online**.

If you currently use alternatives to Word, like LibreOffice Writer or OpenOffice Writer, you can open and view the Word (*.docx*) files used and produced in this project; however, the hidden message will most likely be compromised, as discussed in "**Detecting the Hidden Message**" on **page 119**.

The Strategy

You're an accountant with a beginner's knowledge of Python, and you work for a very intelligent and suspicious man. The project you work on is highly proprietary, with controls—such as email filters—to maintain confidentiality. And if you manage to sneak out a message, a thorough investigation will surely follow. So, you need to hide a clearly suspicious message in an email, either directly or as an attachment, yet evade initial detection and later internal audits.

Here are some constraints:

- You can't send the message directly to the competing corporation, only to an intermediary.
- You need to scramble the message to evade the email filters that will search for keywords.
- You need to hide the encrypted message from sight so as not to arouse suspicion.

The intermediary would be easy to set up, and free encryption sites are easy to find on the internet—but the last item is more problematic.

Steganography is the answer, but as you saw in the previous chapter, hiding even a short message in a null cipher is no easy task. Alternative techniques involve shifting lines of text vertically or words horizontally by small amounts, changing the length of letters, or using the file's metadata —but you're an accountant with limited knowledge of Python and even less time. If only there were an easy way, like invisible ink in the old days.

Creating Invisible Ink

Invisible ink, in this age of electronic ink, might be just crazy enough to work! An invisible font would easily foil a visual perusal of online documents and won't even exist in paper printouts. Since the contents would be encrypted, digital filters looking for keywords like *bid* or the Spanish

names of the producing oil basins would find nothing. And best of all, invisible ink is easy to use—just set the foreground text to the background color.

Formatting text and changing its color requires a word processor like Microsoft Word. To make invisible electronic ink in Word, you just need to select a character, word, or line and set the font color to white. The recipient of the message would then need to select the whole document and use the Highlighter tool (see **Figure 6-4**) to paint the selected text black, thus concealing the standard black letters and bringing the hidden white letters into view.

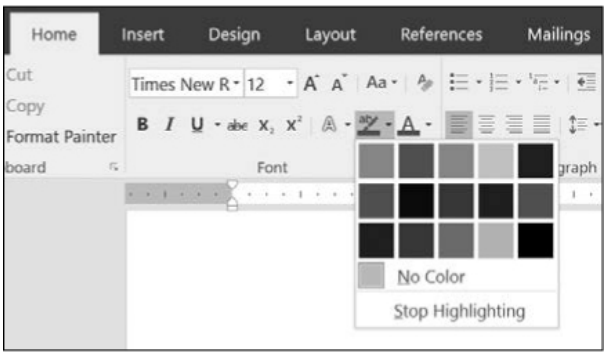


Figure 6-4: The Text Highlight Color tool in Word 2016

Just selecting the document in Word won't reveal the white text (**Figure 6-5**), so someone would have to be very suspicious indeed to find these hidden messages.

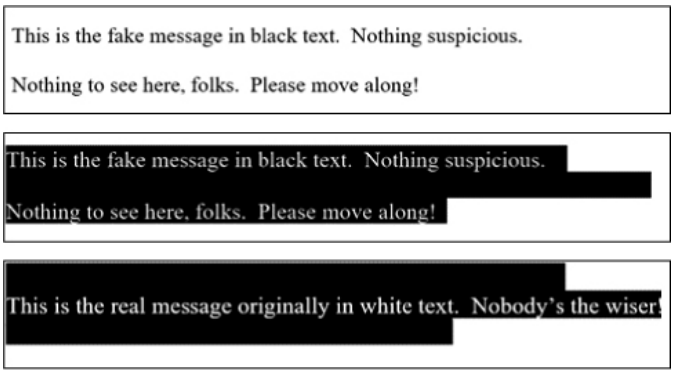


Figure 6-5: Top: a portion of a Word document with the fake message visible; middle: the document selected with CTRL-A; bottom: the real message revealed using the Highlighter tool with the highlight color set to black

Of course, you can accomplish all this in a word processor alone, but there are two cases where a Pythonic approach is preferable: 1) when you have to encrypt a long message and don't want to manually insert and hide all the lines and 2) when you need to send more than a few messages. As you'll see, a short Python program will greatly simplify the process!

Considering Font Types, Kerning, and Tracking

Placing the invisible text is a key design decision. One option is to use the spaces between the visible words of the fake message, but this could trigger spacing-related issues that would make the final product look suspicious.

Proportional fonts use variable character widths to improve readability. Example fonts are Arial and Times New Roman. *Monospace fonts* use a constant character width to support the alignment of text and the recognition of individual characters, especially thin ones such as the (or { characters. As a result, monospace fonts are popular in programming interfaces. Example fonts are Consolas and Courier New.

Kerning is a typographical process for adjusting the spacing and overlap between individual character glyphs in order to improve their visual appeal. A process called *tracking* is used to adjust the character spacing across entire lines or blocks of text for the same purpose. These adjustments aid legibility and readability, ensuring that letters aren't so close together that they're indistinguishable or so far apart that words aren't recognizable. Note that we read words, not letters. If you doubt it, read this: peopl raed wrds nt lttrs. Of corase, contxt hlps.

Kerning between pairs of letters is performed first, followed by tracking, during which the relative kerning of the letter pairs is preserved. As mentioned earlier, these variable widths and automatic corrections can cause problems when you're trying to hide characters between words that use proportional fonts:

To a great mind nothing is little.	*Proportional font with no hidden letters*
To a great mind nothing is little.	*Proportional font with hidden letters between words*
To$a3great!mind2nothingKis little.	*Hidden letters revealed ($3.2K)*

`To a great mind nothing is little.`	*Monospace font with no hidden letters*
`To a great mind nothing is little.`	*Monospace font with hidden letters between words.*
`To$a3great!mind2nothingKis little.`	*Hidden letters revealed ($3.2K)*

If you use a monospace font, the consistent spacing provides a convenient hiding place. But since professional correspondence is more likely to use proportional fonts, the invisible ink technique should focus on the more easily controlled spaces between lines.

Using empty lines between paragraphs is the easiest method to program and to read, and it shouldn't require a long fake message because you can summarize the salient points of a bid succinctly. This is important since you don't want empty pages appended to your visible fake message. Consequently, the footprint for your hidden message should be smaller than for your fake one.

Avoiding Issues

When you're developing software, a good question to ask repeatedly is "How can the user screw this up?" One thing that can go wrong here is that the encryption process will change the letters in your hidden message so that kerning and tracking adjustments may push a word past the line break, causing an automatic line wrap. This will result in uneven and suspicious-looking spaces between paragraphs in the fake message. One way to avoid this is to press ENTER a little early as you're typing in each line of the real message. This will leave some space at the end of the line to accommodate changes due to encryption. Of course, you'll still need to verify the results. Assuming code works is as risky as assuming James Bond is dead!

Manipulating Word Documents with python-docx

A free third-party module called `python-docx` allows Python to manipulate Microsoft Word (*.docx*) files. To download and install the third-party modules mentioned in this book, you'll use the Preferred Installer Program (pip), a package management system that makes it easy to install Python-based software. For Python 3 on Windows and macOS, versions 3.4 and later come with pip preinstalled; for Python 2, pip preinstallation starts with version 2.7.9. Linux users may have to install pip separately. If you find you need to install or upgrade pip, see the instructions at **https://pip.pypa.io/en/stable/installing/** or do an online search on installing pip on your particular operating system.

With the pip tool, you can install `python-docx` by simply running **`pip install python-docx`** in a command, PowerShell, or terminal window, depending on your operating system. Online instructions for `python-docx` are available at **https://python-docx.readthedocs.io/en/latest/**.

For this project, you need to understand the `paragraph` and `run` objects. The `python-docx` module organizes data types using three objects in the following hierarchy:

- `document`: The whole document with a list of `paragraph` objects
- `paragraph`: A block of text separated by the use of the ENTER key in Word; contains a list of `run` object(s)
- `run`: A connected string of text with the same *style*

A `paragraph` is considered a *block-level* object, which `python-docx` defines as follows: "a block-level item flows the text it contains between its left and right edges, adding an additional line each time the text extends beyond its right boundary. For a `paragraph` object, the boundaries are generally the page margins, but they can also be column boundaries if the page is laid out in columns, or cell boundaries if the `paragraph` occurs inside a table cell. A table is also a block-level object."

A `paragraph` object has a variety of attributes that specify its placement within a container—typically a page—and the way it divides its contents into separate lines. You can access the formatting attributes of a `paragraph` with the `ParagraphFormat` object available through the `ParagraphFormat` attribute

of the `paragraph`, and you can set all the `paragraph` attributes using a *paragraph style grouping* or apply them directly to a `paragraph`.

A `run` is an *inline-level* object that occurs within paragraphs or other block-level objects. A `run` object has a read-only `font` attribute providing access to a `font` object. A `font` object provides attributes for getting and setting the character formatting for that `run`. You'll need this feature for setting your hidden message's text color to white.

Style refers to a collection of attributes in Word for paragraphs and characters (`run` objects) or a combination of both. Style includes familiar attributes such as font, color, indention, line spacing, and so on. You may have noticed groupings of these displayed in the Styles pane on Word's Home ribbon (see **Figure 6-6**). Any change in style—to even a single letter—requires the creation of a new `run` object. Currently, only styles that are in the opened *.docx* file are available. This may change in future versions of `python-docx`.

Figure 6-6: The Styles pane in Microsoft Word 2016

You can find full documentation on the use of styles in `python-docx` at **http://python-docx.readthedocs.io/en/latest/user/styles-using.html**.

Here's an example of paragraphs and runs as `python-docx` sees them:

I am a single paragraph of one run because all my text is the same style.

I am a single paragraph with two runs. **I am the second run because my style changed to bold.**

I am a single paragraph with three runs. **I am the second run because my style changed to bold. The third run is my last** *word.*

If any of this seems unclear, don't fret. You don't need to know `python-docx` in any detail. As with any piece of code, you mainly need to know *what you want to do*. An online search should yield plenty of useful suggestions and complete samples of code.

NOTE

For this to work smoothly, don't change styles within the real (hidden) message and make sure you end every line in a hard return by manually pressing the ENTER key. Unfortunately, Word doesn't have a special character for soft returns caused by automatic word wrapping. So, you can't go into an existing Word document with automatic line breaks and use Find and Replace to change them all to hard returns. Such is the life of a mole.

Downloading the Assets

The external files you'll need are downloadable from **https://www.nostarch.com/impracticalpython/** and should be saved in the same folder as the code:

template.docx An empty Word doc formatted with official Holmes Corporation styles, fonts, and margins

fakeMessage.docx The fake message, without letterhead and date, in a Word document

realMessage.docx The real message in plaintext, without letterhead and date, in a Word document

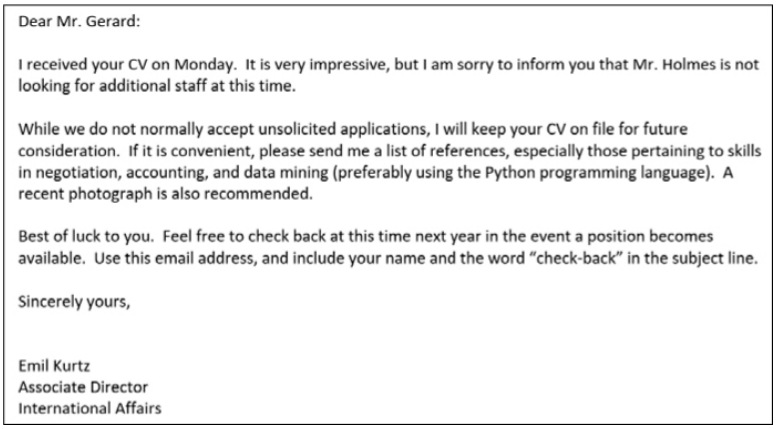
realMessage_Vig.docx The real message encrypted with the Vigenère cipher

example_template_prep.docx An example of the fake message used to create the template document (the program doesn't require this file to run)

NOTE

If you're using Word 2016, an easy way to make a blank template file is to write the fake message (including letterhead) and save the file. Then delete all the text and save the file again with a different name. When you assign this blank file to a variable with `python-docx`, all the existing styles will be retained. Of course, you could use a template file with the letterhead already included, but for the purpose of learning more about `python-docx`, we'll build the letterhead here using Python.

Take a moment to view these first four documents in Word. These files comprise the inputs to the *elementary_ink.py* program. The fake and real messages—the second and third items listed—are also shown in **Figures 6-7** and **6-8**.



Dear Mr. Gerard:

I received your CV on Monday. It is very impressive, but I am sorry to inform you that Mr. Holmes is not looking for additional staff at this time.

While we do not normally accept unsolicited applications, I will keep your CV on file for future consideration. If it is convenient, please send me a list of references, especially those pertaining to skills in negotiation, accounting, and data mining (preferably using the Python programming language). A recent photograph is also recommended.

Best of luck to you. Feel free to check back at this time next year in the event a position becomes available. Use this email address, and include your name and the word "check-back" in the subject line.

Sincerely yours,

Emil Kurtz
Associate Director
International Affairs

Figure 6-7: The "fake" text in the fakeMessage.docx *file*

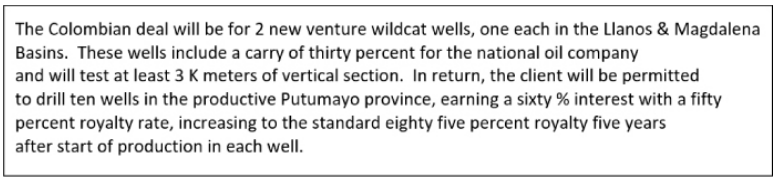


Figure 6-8: The real message in the realMessage.docx *file*

Note that the real message contains some numbers and special characters. These won't be encrypted with the Vigenère tableau we'll use, and I've included them to make that point. Ideally, they would be spelled out (for example, "three" for "3" and "percent" for "%") for maximum secrecy when we add the Vigenère cipher later.

The Pseudocode

The following pseudocode describes how to load the two messages and the template document, interleave and hide the real message in blank lines using a white font, and then save the hybrid message.

```
Build assets:
In Word, create an empty doc with desired formatting/styles (template)
In Word, create an innocuous fake message that will be visible & have enough     blank lines to hol
In Word, create the real message that will be hidden
Import docx to allow manipulation of Word docs with Python
Use docx module to load the fake & real messages as lists
Use docx to assign the empty doc to a variable
Use docx to add letterhead banner to empty doc
Make counter variable for lines in real message
Define function to format paragraph spacing with docx
For line in fake message:
    If line is blank and there are still lines in the real message:
        Use docx & counter to fill blank with line from real message
        Use docx to set real message font color to white
        Advance counter for real message
    Otherwise:
        Use docx to write fake line
    Run paragraph spacing function
Use docx to save final Word document
```

The Code

The *elementary_ink.py* program in **Listing 6-1** loads the real message, the fake message, and the empty template document. It hides the real message in the blank lines of the fake message using a white font, and then saves the hybrid message as an innocuous and professional-looking piece of correspondence that can be attached to an email. You can download the code from **https://www.nostarch.com/impracticalpython/**.

Importing python-docx, Creating Lists, and Adding a Letterhead

Listing 6-1 imports `python-docx`, turns the lines of text in the fake and real messages into list items, loads the template document that sets the styles, and adds a letterhead.

elementary_ink.py, part 1

```
   import docx
❶ from docx.shared import RGBColor, Pt

❷ # get text from fake message & make each line a list item
   fake_text = docx.Document('fakeMessage.docx')
   fake_list = []
   for paragraph in fake_text.paragraphs:
       fake_list.append(paragraph.text)

❸ # get text from real message & make each line a list item
   real_text = docx.Document('realMessage.docx')
   real_list = []
   for paragraph in real_text.paragraphs:
    ❹ if len(paragraph.text) != 0:  # remove blank lines
           real_list.append(paragraph.text)

❺ # load template that sets style, font, margins, etc.
   doc = docx.Document('template.docx')

❻ # add letterhead
   doc.add_heading('Morland Holmes', 0)
   subtitle = doc.add_heading('Global Consulting & Negotiations', 1)
   subtitle.alignment = 1
   doc.add_heading('', 1)
❼ doc.add_paragraph('December 17, 2015')
   doc.add_paragraph('')
```

Listing 6-1: Imports `python-docx`, loads important .docx *files, and adds a letterhead*

After importing the `docx` module—not as "python-docx"—use `docx.shared` to gain access to the color (`RGBColor`) and length (`Pt`) objects in the `docx` module ❶. These will allow you to change the font color and set the spacing between lines. The next two code blocks load the fake ❷ and real ❸ message Word documents as lists. Where the ENTER key was pressed in each Word document determines what items will be in these lists. For the real message to be hidden, remove any blank lines so that your message will be as short as possible ❹. Now you can use list indexes to merge the two messages and keep track of which is which.

Next, load the template document that contains the preestablished styles, fonts, and margins ❺. The `docx` module will write to this variable and ultimately save it as the final document.

With the inputs loaded and prepped, format the letterhead of the final document to match that of the Holmes Corporation ❻. The `add_heading()` function adds a heading style paragraph with text and integer arguments. Integer `0` designates the highest-level heading, or Title style, inherited from the template document. The subtitle is formatted with `1`, the next heading style available, and is center aligned, again with the integer `1` (`0` =

left justified, `2` = right justified). Note that, when you add the date, you don't need to supply an integer ❼. When you don't provide an argument, the default is to inherit from the existing style hierarchy, which in the template is left justified. The other statements in this block just add blank lines.

Formatting and Interleaving the Messages

Listing 6-2 does the real work, formatting the spacing between lines and interleaving the messages.

elementary_ink.py, part 2

```
❶ def set_spacing(paragraph):
      """Use docx to set line spacing between paragraphs."""
      paragraph_format = paragraph.paragraph_format
      paragraph_format.space_before = Pt(0)
      paragraph_format.space_after = Pt(0)

❷ length_real = len(real_list)
  count_real = 0  # index of current line in real (hidden) message

  # interleave real and fake message lines
  for line in fake_list:
   ❸ if count_real < length_real and line == "":
       ❹ paragraph = doc.add_paragraph(real_list[count_real])
       ❺ paragraph_index = len(doc.paragraphs) - 1
          # set real message color to white
          run = doc.paragraphs[paragraph_index].runs[0]
          font = run.font
       ❻ font.color.rgb = RGBColor(255, 255, 255) # make it red to test
       ❼ count_real += 1
      else:
       ❽ paragraph = doc.add_paragraph(line)

    ❾ set_spacing(paragraph)

❿ doc.save('ciphertext_message_letterhead.docx')

  print("Done")
```

Listing 6-2: Formats paragraphs and interleaves lines of fake and real messages

Define a function that formats the spacing between paragraphs using `python-docx`'s `paragraph_format` attribute ❶. Line spacing before and after the hidden line is set to `0` points to ensure that the output doesn't have suspiciously large gaps between paragraphs, like the ones on the left-hand side of **Figure 6-9**.

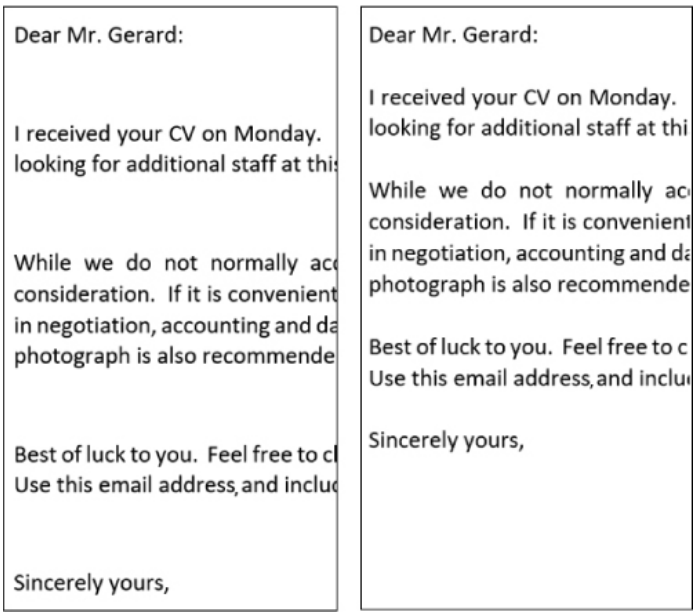


Figure 6-9: Fake message line spacing without `python-docx` paragraph formatting (left) vs. with formatting (right)

Next, define the working space by getting the length of the list that holds the real message ❷. Remember that the hidden real message needs to be shorter than the visible fake message so that there are sufficient blank lines to hold it. Follow this by initiating a counter. The program will use it to keep track of which line (list item) it's currently processing in the real message.

Since the list made from the fake message is the longest and sets the dimensional space for the real message, loop through the fake message using two conditionals: 1) whether you've reached the end of the real message and 2) whether a line in the fake list is blank ❸. If there are still real message lines and the fake message line is blank, use `count_real` as an index for `real_list` and use `python-docx` to add it to the document ❹.

Get the index of the line you just added by taking the length of `doc.paragraphs` and subtracting 1 ❺. Then use this index to set the real message line to a `run` object (it will be the first `run` item `[0]` in the list, as the real message uses a single style) and set its font color to white ❻. Since the program has now added a line from the real list in this block, the `count_real` counter advances by 1 ❼.

The subsequent `else` block addresses the case where the line chosen from the fake list in the `for` loop isn't empty. In this case, the fake message line is added directly to the paragraph ❽. Finish the `for` loop by calling the line spacing function, `set_spacing()` ❾.

Once the length of the real message has been exceeded, the `for` loop will continue to add the remainder of the fake message—in this case, Mr. Kurtz's signature info—and save the document as a Word *.docx* file in the final line ❿. Of course, in real life, you'd want to use a less suspicious filename than *ciphertext_message_letterhead.docx*!

Note that, because you're using a `for` loop based on the fake message, appending any more hidden lines after the `for` loop ends—that is, after you

reach the end of the items in the fake list—is impossible. If you want more space, you must enter hard returns at the bottom of the fake message, but be careful not to add so many that you force a page break and create a mysterious empty page!

Run the program, open the saved Word document, use CTRL-A to select all the text, and then set the Highlight color (see **Figure 6-4**) to dark gray to see both messages. The secret message should be revealed (**Figure 6-10**).

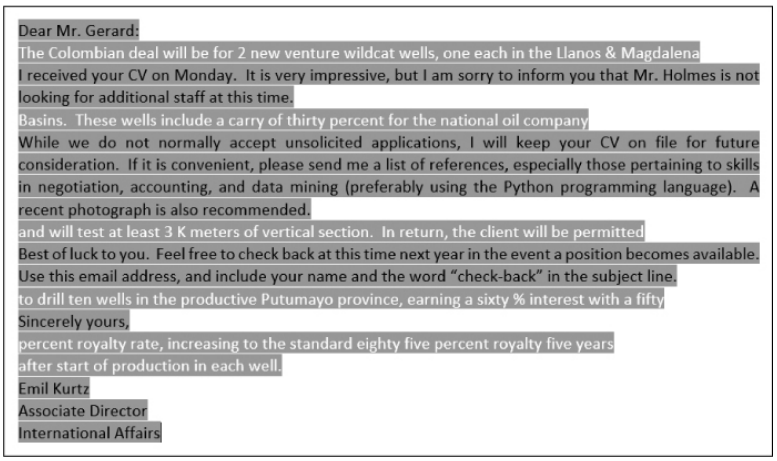

Dear Mr. Gerard:
The Colombian deal will be for 2 new venture wildcat wells, one each in the Llanos & Magdalena
I received your CV on Monday. It is very impressive, but I am sorry to inform you that Mr. Holmes is not looking for additional staff at this time.
Basins. These wells include a carry of thirty percent for the national oil company
While we do not normally accept unsolicited applications, I will keep your CV on file for future consideration. If it is convenient, please send me a list of references, especially those pertaining to skills in negotiation, accounting, and data mining (preferably using the Python programming language). A recent photograph is also recommended.
and will test at least 3 K meters of vertical section. In return, the client will be permitted
Best of luck to you. Feel free to check back at this time next year in the event a position becomes available. Use this email address, and include your name and the word "check-back" in the subject line.
to drill ten wells in the productive Putumayo province, earning a sixty % interest with a fifty
Sincerely yours,
percent royalty rate, increasing to the standard eighty five percent royalty five years
after start of production in each well.
Emil Kurtz
Associate Director
International Affairs

Figure 6-10: Word document highlighted in dark gray to show both the fake message and the unencrypted real message

Adding the Vigenère Cipher

The code so far uses the plaintext version of the real message, so anyone who changes the document's highlight color will be able to read and understand the sensitive information in it. Since you know Mr. Kurtz encrypted this using the Vigenère cipher, go back and alter the code to replace the plaintext with the encrypted text. To do this, find the following line:

```
real_text = docx.Document('realMessage.docx')
```

This line loads the real message as plaintext, so change the filename to the one shown here in bold:

```
real_text = docx.Document('realMessage_Vig.docx')
```

Rerun the program and again reveal the hidden text by selecting the whole document and setting the Highlight color to dark gray (**Figure 6-11**).

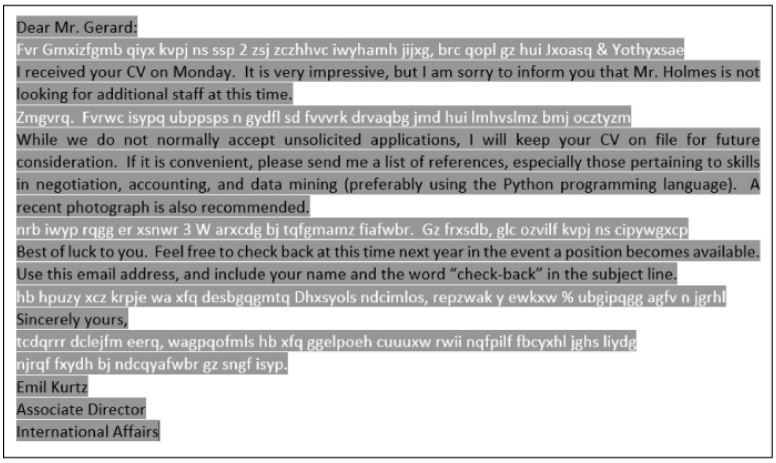


Figure 6-11: Word document highlighted in dark gray to show both the fake message and the encrypted real message

The secret message should be visible but unreadable to anyone who cannot interpret the cipher. Compare the encrypted message in **Figure 6-11** to the unencrypted version in **Figure 6-10**. Note that numbers and the % sign occur in both versions. These were retained to demonstrate the potential pitfalls related to the encryption choice. You would want to augment the Vigenère cipher to include these characters—or just spell them out. That way, even if your message is discovered, you leave as few clues as possible as to the subject matter.

If you want to encode your own message with the Vigenère cipher, do an internet search for "online Vigenère encoder." You'll find multiple sites, such as **http://www.cs.du.edu/~snarayan/crypt/vigenere.html**, that let you type or paste in plaintext. And if you want to write your own Python program for encrypting with the Vigenère cipher, see *Cracking Codes with Python* (No Starch Press, 2018) by Al Sweigart.

If you play around with your own real messages, encrypted or not, make sure you're using the same font as in the fake message. A font is both a typeface, like Helvetica Italic, and a size, such as 12. Remember from "**Considering Font Types, Kerning, and Tracking**" on **page 109** that if you try to mix fonts, especially proportional and monospace fonts, the hidden message lines may wrap, resulting in uneven spacing between paragraphs of the real message.

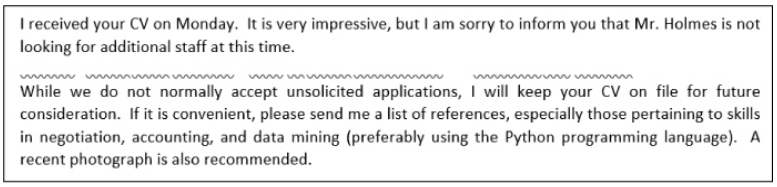
Detecting the Hidden Message

Could Joan Watson, or any other detective, have found your hidden message quickly? The truth is, probably not. In fact, as I write these words, I am watching an episode of *Elementary* where Joan is busy investigating a company by reading through a box of email printouts! The use of the Vigenère cipher may have been just a bit of lazy writing in an overall intelligently crafted series. Still, we can speculate on what might give you away.

Since the final bid was probably not sent until close to the bid date, the search could be limited to correspondence sent *after* the bid was finalized, thereby eliminating a lot of noise. Of course, a detective won't know exactly what they're looking for—or even if there *is* a mole—which leaves

a large search space. And there's always the possibility that the information was passed in a phone conversation or clandestine meeting.

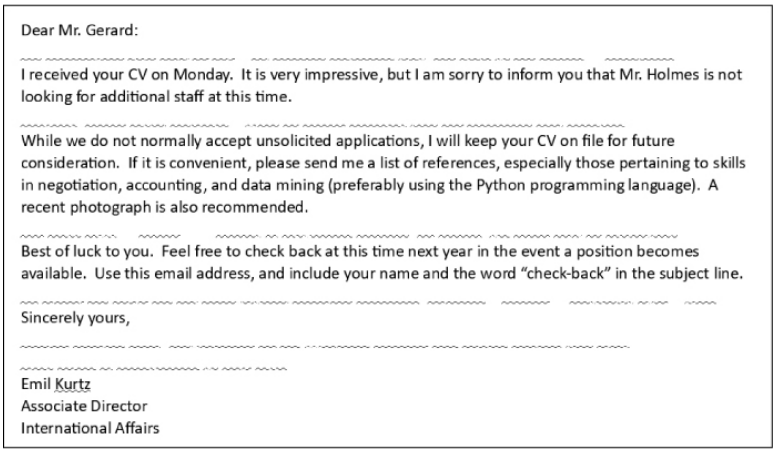
Assuming there was a manageable volume of email and a hidden-message hypothesis was being pursued, an investigator might detect your invisible ink in several ways. For example, the Word spellchecker will not flag the white, nonsensical encrypted words as long as they haven't been made visible. If, as a check, you swiped and reset the font color on some of the hidden words, they will be permanently compromised, even after their color has been restored to white. The spellchecker will underline them with an incriminating red squiggly line (see **Figure 6-12**).


I received your CV on Monday. It is very impressive, but I am sorry to inform you that Mr. Holmes is not looking for additional staff at this time.

While we do not normally accept unsolicited applications, I will keep your CV on file for future consideration. If it is convenient, please send me a list of references, especially those pertaining to skills in negotiation, accounting, and data mining (preferably using the Python programming language). A recent photograph is also recommended.

Figure 6-12: Previously revealed invisible encrypted words underlined by the Word Spelling and Grammar tool

If the investigating detective uses an alternative to Word to open the document, the product's spellchecker will most likely reveal the hidden words (see **Figure 6-13**). This risk is mitigated somewhat by the dominance of Microsoft Word in the marketplace.


Dear Mr. Gerard:

I received your CV on Monday. It is very impressive, but I am sorry to inform you that Mr. Holmes is not looking for additional staff at this time.

While we do not normally accept unsolicited applications, I will keep your CV on file for future consideration. If it is convenient, please send me a list of references, especially those pertaining to skills in negotiation, accounting, and data mining (preferably using the Python programming language). A recent photograph is also recommended.

Best of luck to you. Feel free to check back at this time next year in the event a position becomes available. Use this email address, and include your name and the word "check-back" in the subject line.

Sincerely yours,

Emil Kurtz
Associate Director
International Affairs

Figure 6-13: The spellchecker in LibreOffice Writer will highlight the invisible words.

Second, using CTRL-A to highlight all the text within Word won't reveal the hidden text, but it would indicate that some blank lines are longer than others (see **Figure 6-14**), suggesting to the very observant that something is amiss.

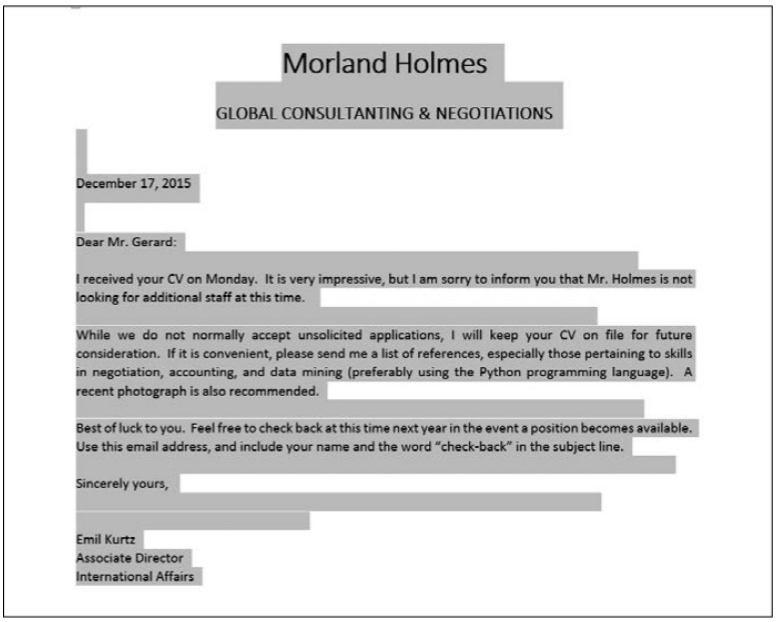


Figure 6-14: Selecting the whole Word document reveals differences in the length of blank lines.

Third, opening the Word document using the preview functionality in some email software may reveal the hidden text when the contents are selected through swiping or using CTRL-A (**Figure 6-15**).

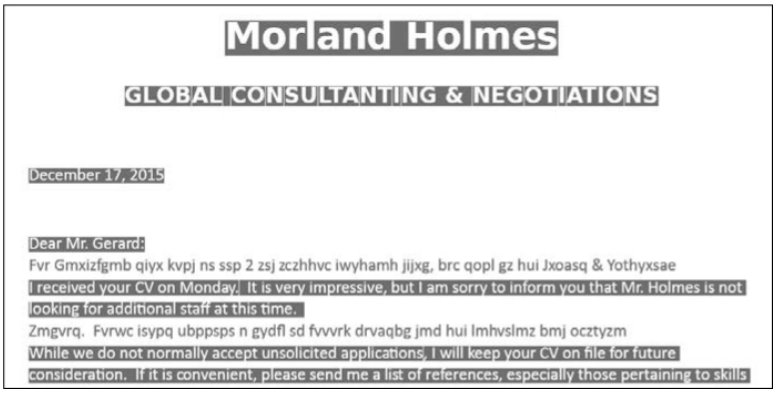


Figure 6-15: Selecting the whole document in the Yahoo! Mail Preview panel reveals the hidden text.

But while selecting hidden text in the Yahoo! Mail Preview panel reveals the text, the same is not true in the Microsoft Outlook Preview panel in **Figure 6-16**.

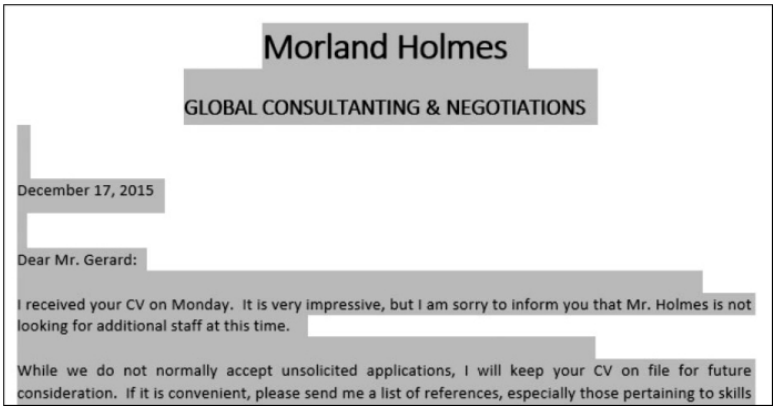


Figure 6-16: Selecting the whole document in the Microsoft Outlook Preview panel does not reveal the hidden text.

Finally, saving the Word document as a plain text (**.txt*) file would remove all formatting and leave the hidden text exposed (**Figure 6-17**).

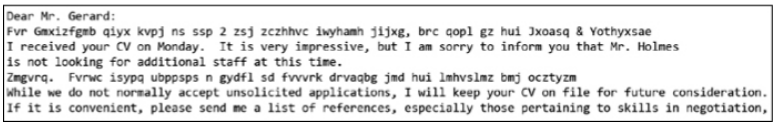


Figure 6-17: Saving the Word document as a plain text (.txt) file reveals the hidden text.*

To conceal a secret message with steganography, you have to conceal not only the *contents* of the message but also the fact that a message even *exists*. Our electronic invisible ink can't always guarantee this, but from a mole's point of view, the weaknesses just listed involve either them making a mistake, which could theoretically be controlled, or an investigator taking a dedicated and unlikely action, such as swiping text, saving files in a different format, or using a less-common word processor. Assuming the mole in *Elementary* considered these acceptable risks, electronic invisible ink provides a plausible explanation for why the internal company investigation failed.

Summary

In this chapter, you used steganography to hide an encrypted message within a Microsoft Word document. You used a third-party module, called `python-docx`, to directly access and manipulate the document using Python. Similar third-party modules are available for working with other popular document types, like Excel spreadsheets.

Further Reading

You can find online documentation for `python-docx` at **https://python-docx.readthedocs.io/en/latest/** and **https://pypi.python.org/pypi/python-docx**.

Automate the Boring Stuff with Python (No Starch Press, 2015) by Al Sweigart, covers modules that allow Python to manipulate PDFs, Word files, Excel spreadsheets, and more. **Chapter 13** contains a useful tutorial on `python-docx`, and the appendix covers installing third-party modules with pip.

You can find beginner-level Python programs for working with ciphers in *Cracking Codes with Python* (No Starch Press, 2018) by Al Sweigart.

Mysterious Messages (The Penguin Group, 2009) by Gary Blackwood is an interesting and well-illustrated history of steganography and cryptography.

Practice Project: Checking the Number of Blank Lines

Improve the hidden message program by writing a function that compares the number of blank lines in the fake message to the number of lines in the real message. If there is insufficient space to hide the real message, have the function warn the user and tell them how many blank lines to add to the fake message. Insert and call the function in a copy of

the *elementary_ink.py* code, just before loading the template document. You can find a solution in the appendix and online at **https://www.nostarch.com/impracticalpython/** in *elementary_ink_practice.py*. For testing, download *realMessageChallenge.docx* from the same site and use it for the real message.

Challenge Project: Using Monospace Font

Rewrite the *elementary_ink.py* code for monospace fonts and hide your own short message in the spaces between words. See "**Considering Font Types, Kerning, and Tracking**" on **page 109** for a description of monospace fonts. As always with challenge projects, no solution is provided.